

Data Quality & Governance in Databricks

Best Practices for Medallion Architecture

Data Quality in Databricks

Databricks provides multiple approaches for implementing data quality checks through Delta Lake and Unity Catalog. The platform enables both reactive and proactive quality management strategies.

Delta Lake Quality Features

- **Schema enforcement:** Prevents incompatible data writes
- **Schema evolution:** Controlled schema changes with ALTER TABLE
- **Time travel:** Query historical versions for root cause analysis
- **ACID transactions:** Ensures data consistency across operations

Quality Implementation Tools

- **Delta Live Tables (DLT) with expectations:** Declarative quality rules with automated enforcement
- **Great Expectations:** Python-based validation framework for complex checks
- **Custom PySpark frameworks:** Flexible validation logic in ETL pipelines
- **SQL constraints:** CHECK constraints and NOT NULL enforcement

Unity Catalog Governance

Unity Catalog provides enterprise-grade governance capabilities with centralized metadata management, fine-grained access control, and automated lineage tracking across all workspace assets.

Core Governance Features

- **Three-level namespace:** Catalog > Schema > Table hierarchy for logical organization
- **Automated lineage:** Column-level tracking for impact analysis and compliance
- **Data classification:** Tag PII, PHI, and sensitive data for regulatory compliance
- **RBAC & attribute-based access:** Grant permissions at catalog, schema, or table level
- **Audit logging:** Complete access history for security and compliance reviews
- **Row and column filters:** Dynamic data masking based on user identity

Governance Best Practices

- Separate catalogs for Bronze, Silver, and Gold layers
- Apply tags at schema creation for consistent classification
- Use service principals for pipeline automation with least privilege
- Enable audit logs and monitor access patterns regularly
- Document ownership and stewardship responsibilities in table properties

Medallion Architecture Quality Checks

The Bronze-Silver-Gold architecture requires layer-specific quality controls. Each layer has distinct validation requirements aligned with its data maturity level.

Bronze Layer: Raw Data Ingestion

Check Type	Implementation
Schema Validation	Enforce expected schema on ingestion, reject files with missing columns or wrong data types
Null Detection	Flag records with nulls in critical fields (primary keys, timestamps)
Duplicate Detection	Identify duplicate records using composite keys, count occurrences
File Metadata	Track source file name, ingestion timestamp, record count per batch

Silver Layer: Cleansed & Conformed

Check Type	Implementation
Data Type Validation	Verify date formats, numeric ranges, string patterns using regex
Business Rules	Apply domain logic: age > 0, order_date <= ship_date, status in valid values
Referential Integrity	Foreign key validation against dimension tables, orphan record detection
Standardization	Normalize country codes, phone formats, currency values to standard formats

Gold Layer: Business Aggregations

Check Type	Implementation
KPI Validation	Compare calculated metrics against expected thresholds and historical trends
Aggregate Completeness	Ensure all expected time periods and dimensions are present in aggregations
Reconciliation	Row count and sum checks between Silver and Gold, variance analysis
Outlier Detection	Statistical checks for anomalies using standard deviation or IQR methods

Data Quality Dimensions (DQD) Framework

Comprehensive quality management requires monitoring six critical dimensions. Each dimension addresses specific aspects of data reliability and trustworthiness.

1. Completeness

Definition: Measures whether all expected data is present

Key Checks

- Null rate analysis: Count nulls per column, flag if > threshold (e.g., 5%)
- Record count validation: Compare expected vs actual row counts by partition
- Mandatory field checks: Ensure critical business fields are populated

2. Accuracy

Definition: Verifies data correctly represents real-world values

Key Checks

- Data type validation: Verify numeric fields contain numbers, dates are valid dates
- Range checks: Amount > 0, percentage between 0-100, dates within valid window
- Format validation: Email regex, phone number patterns, ISO country codes

3. Consistency

Definition: Ensures data is uniform across tables and systems

Key Checks

- Cross-table validation: Customer totals match between orders and payments tables
- Referential integrity: All foreign keys exist in parent tables
- Business logic consistency: Order status transitions follow valid state machine

4. Timeliness

Definition: Data arrives and updates within expected time windows

Key Checks

- Freshness monitoring: Alert if latest data timestamp exceeds SLA (e.g., 4 hours old)
- Pipeline latency: Track time between source update and warehouse availability
- Batch completeness: Verify all expected daily batches arrived on schedule

5. Uniqueness

Definition: Records are not duplicated inappropriately

Key Checks

- Primary key uniqueness: Count distinct vs total rows, flag if mismatched
- Duplicate detection: Identify records with identical business keys
- Near-duplicate analysis: Fuzzy matching on name fields to find potential duplicates

6. Validity

Definition: Data conforms to defined business rules and constraints

Key Checks

- Enumeration checks: Status values exist in allowed list (PENDING, APPROVED, REJECTED)

- Relationship validation: Quantity ordered $\leq$ quantity in stock for inventory systems
- Domain-specific rules: Credit limit $\geq$ outstanding balance, age $\geq$ 18 for accounts

Implementation Patterns

Delta Live Tables with Expectations

DLT provides declarative quality enforcement through expectations. Use `@expect` for warnings and `@expect_or_fail` for hard stops.

Example - Bronze to Silver validation:

```
@dlt.table @dlt.expect_or_fail("valid_timestamp", "event_time IS NOT NULL")
@dlt.expect("valid_amount", "amount > 0") def silver_orders():      return
spark.readStream.table("bronze_orders")
    .filter(col("_rescued_data").isNull())
```

Quarantine Table Pattern

Route invalid records to separate quarantine tables for investigation rather than blocking pipelines. Maintain audit trail with validation failure reasons.

Implementation approach:

- Add validation columns: `is_valid` (boolean), `validation_errors` (array of strings)
- Write failed records to quarantine schema with original data + error details
- Schedule periodic review with data stewards for root cause analysis
- Implement retry logic after upstream fixes are deployed

Monitoring & Alerting Strategy

Key Metrics to Track

- Record counts by table and partition (daily, hourly as needed)
- Null percentage for critical columns with thresholds
- Pipeline execution duration and failure rates
- Quarantine table size trends indicating upstream issues
- Data freshness lag from source systems

Alert Configuration

- Critical: Pipeline failures, missing batches, breach of SLA thresholds
- Warning: Elevated null rates, duplicate increases, slow query performance
- Info: Successful completions, data volume changes beyond normal variance

Best Practices Checklist

- Document all quality rules in Unity Catalog table properties
- Version control validation logic alongside pipeline code
- Implement data quality dashboards in Databricks SQL for visibility
- Establish data quality SLAs with business stakeholders
- Automate validation report generation and distribution
- Test quality rules in development before production deployment
- Conduct regular data quality audits and trend analysis
- Maintain runbook for handling common data quality incidents